PII Scanner Agent — MSI Installer Build Guide

Repo: https://github.com/SChinmaya15/PiiScannerAgent **Target:** Windows Service (`PiiScanner.exe`) installed via a self-contained `.msi` **Tooling:** .NET 8 SDK + WiX Toolset v5/v6 (free, scriptable, no Visual Studio required)

> This entire process must run on a **Windows machine** (or a Windows CI agent). MSI files cannot be built on Linux/macOS.

---

1. What's in the repo

```
PiiScannerAgent/
├── PiiScanner.sln
├── PiiScanner/                          ← the Worker Service (net8.0)
|   ├── PiiScanner.csproj
|   ├── Program.cs
|   ├── Worker.cs
|   ├── Scan.cs
|   ├── appsettings.json
|   └── appsettings.Development.json
└── PiiScannerAgent.Validators/          ← class library referenced by
PiiScanner
    ├── PiiScanner.Validators.csproj
    ├── AadhaarValidator.cs
    ├── PanValidator.cs
    ├── EmailValidator.cs
    ├── MobileNumberValidator.cs
    ├── BankAccountValidator.cs
    └── CreditCardValidator.cs
```

`PiiScanner` is built with the `Microsoft.NET.Sdk.Worker` SDK — this is what `dotnet new worker` produces, and it's exactly the kind of app WiX's `ServiceInstall`/`ServiceControl` elements are designed to wrap.

---

2. Prerequisites

Install these on the Windows build machine:

Tool	Install command / link
.NET 8 SDK	https://dotnet.microsoft.com/download/dotnet/8.0
WiX Toolset CLI (global .NET tool)	`dotnet tool install --global wix`
Git	https://git-scm.com/download/win

Verify after installing:

```powershell
dotnet --version        # should print an 8.x SDK version
wix --version            # should print the installed WiX version
```

You do **not** need Visual Studio or the HeatWave extension for the steps below — everything is done with the `dotnet` and `wix` CLIs, which is more reliable to reproduce and to run in CI later. (If you prefer a GUI, see the note at the end of §6.)

---

3. Fix the repo before building

3.1 Remove the broken `PiiScanner.Infrastructure` reference (blocking)

Open `PiiScanner/PiiScanner.csproj`. It currently contains:

```xml
<ItemGroup>
  <Reference Include="PiiScanner.Infrastructure">
    <HintPath>..\..\..\repo\PiiScanner\PiiScanner.Infrastructure\bin\Debug\net8
  </Reference>
</ItemGroup>
```

That `HintPath` points to a folder structure (`..\..\..\repo\...`) that only exists on the original author's machine — it will not exist on yours, and `dotnet build`/`publish` will fail with an "assembly not found" error. A search of every `.cs` file in the repo shows **no code anywhere uses `PiiScanner.Infrastructure`** — it's a leftover, unused reference. Delete the whole `<ItemGroup>` block.

3.2 Recommended: register the host as a real Windows Service

`Program.cs` currently builds a generic host but never tells it that it may run as a Windows Service:

```csharp
```

```
using PiiScanner;
using System.Net.Http;

var builder = Host.CreateApplicationBuilder(args);

builder.Services.AddSingleton(new HttpClient());
builder.Services.AddSingleton<Scan>();
builder.Services.AddHostedService<Worker>();

var host = builder.Build();
host.Run();
```

Without `AddWindowsService()`, the exe will still *run* once installed as a service, but it won't correctly receive Service Control Manager stop/pause signals or log to the Windows Event Log — so `net stop`, restarts, and graceful shutdown become unreliable. Add the package and one line:

powershell

```
dotnet add PiiScanner\PiiScanner.csproj package Microsoft.Extensions.Hosting.Wi
```

csharp

```
using PiiScanner;
using System.Net.Http;

var builder = Host.CreateApplicationBuilder(args);

// Makes the host behave like a proper Windows Service (SCM start/stop, Event L
// This is a no-op when running as a console app, so `dotnet run` still works n
builder.Services.AddWindowsService(options =>
{
    options.ServiceName = "PiiScannerAgent";
});

builder.Services.AddSingleton(new HttpClient());
builder.Services.AddSingleton<Scan>();
builder.Services.AddHostedService<Worker>();

var host = builder.Build();
host.Run();
```

After both fixes, `PiiScanner/PiiScanner.csproj` should look like:

```xml
<Project Sdk="Microsoft.NET.Sdk.Worker">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UserSecretsId>dotnet-PiiScanner-fafff948-8950-4c59-a2d7-24cdc07e824f</User
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="8.0.1" />
    <PackageReference Include="Microsoft.Extensions.Hosting.WindowsServices" Ve
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\PiiScannerAgent.Validators\PiiScanner.Validat
  </ItemGroup>

</Project>
```

Confirm it builds:

```powershell
dotnet build .\PiiScanner.sln -c Release
```

---

4. Publish the service as a single self-contained exe

Publishing self-contained means the target machine does **not** need the .NET 8 runtime installed — everything is bundled into one `PiiScanner.exe`. This is the simplest thing to ship in an MSI.

From the repo root:

```powershell
```

```
dotnet publish .\PiiScanner\PiiScanner.csproj `
  -c Release `
  -r win-x64 `
  --self-contained true `
  -p:PublishSingleFile=true `
  -p:IncludeNativeLibrariesForSelfExtract=true `
  -p:DebugType=None `
  -o .\publish
```

This produces `.\publish\` containing essentially two files:

- `PiiScanner.exe` (the whole app, bundled)
- `appsettings.json`

(`appsettings.Development.json` is also written there by default — exclude it from the installer; it shouldn't ship to production. Also double check `appsettings.json`'s `AgentRegistration:Url` points at your real registration server before you ship, not `https://localhost:44346/...`.)

> Building for a different CPU? Swap `win-x64` for `win-x86` or `win-arm64`.

---

5. Install the WiX Toolset (if you skipped §2)

powershell

```
dotnet tool install --global wix
wix --version
```

No extensions are required — `ServiceInstall` and `ServiceControl` (the elements that register the Windows Service) are part of WiX's core schema as of v4+, so a plain `wix build` is enough.

---

6. Create the WiX installer source

Create an `installer\` folder at the repo root with one file, `Package.wxs`:

xml

```
<?xml version="1.0" encoding="UTF-8"?>

<!-- Manufacturer and Version are passed in from the command line in step 7,
     so this file never needs hand-editing for routine builds. -->
<?define Name = "PII Scanner Agent" ?>
<?define ServiceName = "PiiScannerAgent" ?>
<?define UpgradeCode = "789C55D8-7691-41BD-B2B9-79389A320035" ?>

<Wix xmlns="http://wixtoolset.org/schemas/v4/wxs">
    <Package Name="$(Name)"
             Manufacturer="$(Manufacturer)"
             Version="$(Version)"
             UpgradeCode="$(var.UpgradeCode)"
             Compressed="true">

        <!-- Blocks installing an older MSI over a newer one; allows in-place u
        <MajorUpgrade DowngradeErrorMessage="A later version of [ProductName] i

        <!-- Installs to C:\Program Files\<Manufacturer>\<Name>\ -->
        <StandardDirectory Id="ProgramFiles6432Folder">
            <Directory Id="ROOTDIRECTORY" Name="$(Manufacturer)">
                <Directory Id="INSTALLFOLDER" Name="$(Name)" />
            </Directory>
        </StandardDirectory>

        <DirectoryRef Id="INSTALLFOLDER">

            <!-- The exe, plus the Windows Service registration -->
            <Component Id="ServiceExecutable" Bitness="always64">
                <File Id="PiiScannerExe"
                      Name="PiiScanner.exe"
                      Source="$(var.PublishDir)\PiiScanner.exe"
                      KeyPath="true" />

                <ServiceInstall Id="ServiceInstaller"
                                Type="ownProcess"
                                Name="$(ServiceName)"
                                DisplayName="$(Name)"
                                Description="Scans the endpoint for PII and rep
                                Start="auto"
                                Account="LocalSystem"
                                ErrorControl="normal" />

                <ServiceControl Id="ServiceControl"
                                Name="$(ServiceName)"
                                Start="install"
```

```
                                Stop="both"
                                Remove="uninstall"
                                Wait="true" />
            </Component>

            <!-- Configuration file shipped alongside the exe -->
            <Component Id="AppSettings" Bitness="always64">
                <File Id="AppSettingsJson"
                      Name="appsettings.json"
                      Source="$(var.PublishDir)\appsettings.json"
                      KeyPath="true" />
            </Component>

            <!-- Cleans up the install folder fully on uninstall, including
                 the AgentId that the Worker writes into appsettings.json at ru
            <Component Id="CleanupFolder" Bitness="always64" Guid="*">
                <CreateFolder />
                <RemoveFile Id="RemoveAllFiles" Name="*.*" On="uninstall" />
                <RemoveFolder Id="RemoveInstallFolder" On="uninstall" />
                <RegistryValue Root="HKLM"
                               Key="Software\$(Manufacturer)\$(Name)"
                               Name="Installed"
                               Type="integer"
                               Value="1"
                               KeyPath="yes" />
            </Component>
        </DirectoryRef>

        <Feature Id="MainFeature" Title="$(Name)" Level="1">
            <ComponentRef Id="ServiceExecutable" />
            <ComponentRef Id="AppSettings" />
            <ComponentRef Id="CleanupFolder" />
        </Feature>

    </Package>
</Wix>
```

Notes on choices made here:

- `Account="LocalSystem"` — broad permissions, simplest to get running. If the agent only needs to read the local filesystem/registry and call out over HTTPS, switch to `Account="NetworkService"` for least-privilege.
- `UpgradeCode` is the GUID above — generate your own once (`[guid]::NewGuid()` in PowerShell) and **never change it again** after your first real release; it's what lets future MSIs detect "this machine already has an older version" and upgrade in place instead of conflicting.

- `Bitness="always64"` matches the `win-x64` publish target from §4.

(Prefer a GUI instead of hand-editing XML? Visual Studio + the HeatWave extension gives you an "MSI Package (WiX v4)" project template that edits the same kind of `.wxs` file with IntelliSense. The XML above will drop straight into that project too.)

---

7. Build the MSI

```powershell
wix build .\installer\Package.wxs `
  -arch x64 `
  -d PublishDir="$(Resolve-Path .\publish)" `
  -d Manufacturer="YourCompany" `
  -d Version="1.0.0.0" `
  -out .\installer\PiiScannerAgentSetup.msi
```

Replace `YourCompany` and `1.0.0.0` with your real values. You should end up with:

```
installer\PiiScannerAgentSetup.msi
```

One-shot script

Save this as `build-installer.ps1` at the repo root to run steps 4 and 7 together:

```powershell
```

```powershell
param(
    [string]$Configuration = "Release",
    [string]$Runtime = "win-x64",
    [string]$Manufacturer = "YourCompany",
    [string]$Version = "1.0.0.0"
)

$ErrorActionPreference = "Stop"
$root         = $PSScriptRoot
$publishDir   = Join-Path $root "publish"
$installerDir = Join-Path $root "installer"
$msiPath      = Join-Path $installerDir "PiiScannerAgentSetup.msi"

Write-Host "==> Publishing PiiScanner.exe ($Runtime, $Configuration)..." -Foreg
dotnet publish (Join-Path $root "PiiScanner\PiiScanner.csproj") `
    -c $Configuration `
    -r $Runtime `
    --self-contained true `
    -p:PublishSingleFile=true `
    -p:IncludeNativeLibrariesForSelfExtract=true `
    -p:DebugType=None `
    -o $publishDir

Write-Host "==> Building MSI..." -ForegroundColor Cyan
wix build (Join-Path $installerDir "Package.wxs") `
    -arch x64 `
    -d PublishDir=$publishDir `
    -d Manufacturer=$Manufacturer `
    -d Version=$Version `
    -out $msiPath

Write-Host "==> Done: $msiPath" -ForegroundColor Green
```

Run it with:

```powershell
.\build-installer.ps1 -Manufacturer "YourCompany" -Version "1.0.0.0"
```

---

8. Test the installer

1. **Install** — right-click `PiiScannerAgentSetup.msi` → **Run as administrator** (the service install step needs elevation). For a verbose log if anything goes wrong:

```powershell
msiexec /i PiiScannerAgentSetup.msi /l*v install.log
```

2. **Verify** — open `services.msc` and confirm **PII Scanner Agent** is listed and running. Check `C:\Program Files\YourCompany\PII Scanner Agent\` contains `PiiScanner.exe` and `appsettings.json`.
3. **Check logs** — since you added `AddWindowsService`, events also show up in **Event Viewer → Windows Logs → Application**, source `PiiScannerAgent`.
4. **Uninstall** — via **Settings → Apps**, or:

```powershell
msiexec /x PiiScannerAgentSetup.msi
```

Confirm the service is gone from `services.msc` and the install folder is removed.

---

9. Optional: code-sign the MSI

An unsigned MSI will trigger Windows SmartScreen / "Unknown publisher" warnings. If you have a code-signing certificate:

```powershell
signtool sign /fd SHA256 /tr http://timestamp.digicert.com /td SHA256 /a .\inst
```

`signtool` ships with the Windows SDK.

---

10. Troubleshooting

Symptom	Cause / fix
`dotnet build` fails: "PiiScanner.Infrastructure" not found	You haven't removed the broken `<Reference>` block — see §3.1
`wix build` fails: file not found at `$(var.PublishDir)\PiiScanner.exe`	Run §4 (`dotnet publish`) before §7, and double-check the `-d PublishDir=...` path
Service installs but won't start (Error 1053)	Usually means the exe crashed on startup before reporting to SCM — check Event Viewer; this is far less likely once `AddWindowsService()` is in place (§3.2)

Symptom	Cause / fix
`ICE` validation errors during `wix build`	Run `wix build ... -sice:ICEnnnn` to suppress a specific ICE check, but read the message first — they usually point at a real packaging issue
Installer runs but "Access denied"	You must run the MSI elevated (administrator) — Windows Service registration requires it

Appendix: full directory layout after following this guide

```
PiiScannerAgent/
├── PiiScanner.sln
├── build-installer.ps1
├── PiiScanner/
│   └── ...
├── PiiScannerAgent.Validators/
│   └── ...
├── publish/                          ← generated by `dotnet publish`
│   ├── PiiScanner.exe
│   └── appsettings.json
└── installer/
    ├── Package.wxs
    └── PiiScannerAgentSetup.msi      ← generated by `wix build`
```